

Latent-space transfer between models for efficient reasoning

Briefing and question prep. Every number comes from a run stored in `results/`.

1. The one-paragraph version

A small model reads the user's question. Instead of writing out an answer, it hands what it understood to a much larger model as raw vectors. The large model thinks about those vectors for a few steps, never writing a single word, and hands its conclusion back. The small model then writes the answer. Only a handful of vectors cross between them, so the large model processes a few dozen positions instead of the whole question. On a three-step arithmetic task this takes a 0.12M-parameter model from **0.684 to 0.9995**, which matches what an 8.0M-parameter model scores on its own. Put a randomly initialised model in the large slot instead and the score falls to 0.743, which is how we know the large model is doing the reasoning.

2. Terms used in this document

Token

A chunk of text, roughly a word or word-piece. Models read and write tokens. Everything else here happens between them.

Embedding

The vector a model turns each input token into before doing anything with it. A list of numbers, typically a few hundred to a few thousand long. The length is the model's *width*, written d .

Hidden state

The vector a model holds for each position after it has processed the input. If an embedding is what the model reads, the hidden state is what it currently thinks about that position. There is one per token per layer, and the ones we use are from the final layer.

Residual stream

The running vector that carries information from the bottom of a transformer to the top. Every layer reads it, computes something, and adds the result back. "Projecting into the large model's residual space" means producing vectors of the right width, and the right statistical shape, that the large model's layers will accept as if they had come from its own embeddings.

Logits

The score the model assigns to every possible next token before choosing one. Turning hidden states into logits is the last step of generating text. In this architecture the large model never takes that step.

KV cache

What a model stores after reading text so it does not have to re-read it. Keeping the cache is why the small model reads the question only once, even though it acts twice.

Chain of thought

Getting a model to write out its reasoning in words before answering. It works, but every step of reasoning costs tokens, which costs time and money.

Latent reasoning, or continuous thought

The alternative this work builds on. Rather than decoding a reasoning step into words, feed the model's own hidden state straight back in as the next input. The model reasons in vectors. This is the Coconut method (Hao et al., 2024). A single continuous thought can hold several possible partial answers at once, which written text cannot do since it has to commit to specific words.

Projector

The small learned network that converts vectors from one model's format into another's. Two models trained separately have no reason to represent things the same way, so something has to translate. Ours is a two-layer network with about 40,000 parameters in each direction.

Adapter, or LoRA

A small set of extra weights bolted onto a frozen model so it can learn a new behaviour without retraining it. "Rank 8" describes how small: a few thousand parameters per layer. Ours start at zero, so the model behaves exactly as before until training moves them.

Random initialisation, or "from scratch"

Every weight in a network is a number, and before training those numbers are filled in with small random values. Training then adjusts them until the network does something useful. Training from random initialisation means learning the task from nothing, rather than starting from a model someone else already trained. Both backbones here were built that way, so they share no common ancestry and cannot communicate just because they happened to be trained on overlapping data.

Untrained model

A network left at its random initialisation, never trained at all. It has the right shape but computes nothing meaningful. This is what the control puts in the large model's place, and why the comparison is informative: a noise network of identical size scores 0.743 where the trained one scores 0.9995.

Pretrained

A model already trained on a large corpus before you touch it, such as Qwen2.5-7B. The small-scale experiment avoids these deliberately; the attempts that used them are in section 7 and did not work.

Frozen

Weights held fixed during training. Both backbone models are frozen while the handoff is learned, so any improvement has to come from the pieces connecting them.

Curriculum

Training in stages of increasing difficulty. Here, each stage replaces one more written reasoning step with a vector, until none of the reasoning is written down.

FLOPs

Floating-point operations, a hardware-independent measure of how much arithmetic answering one question takes. Used here instead of wall-clock time because it does not depend on which GPU you happen to own.

Control

A version of the experiment with one ingredient deliberately broken, run to check that the ingredient is what produced the result. Our main control replaces the trained large model with an untrained one of exactly the same shape.

Floor and ceiling

The small model alone, and the large model alone. The handoff is only interesting if it lands well above the floor, and the ceiling tells you how much room there was.

3. What the two models are

Neither is a downloaded model. Both are ordinary decoder-only transformers, the same family as GPT, written from scratch and trained from random initialisation for this experiment. That was a deliberate choice: it means the two models share nothing except the task, so any communication between them has to be genuinely learned rather than inherited from common pretraining.

	SMALL MODEL	LARGE MODEL
parameters	0.12M	7.99M
layers	2	10
width (d)	64	256
attention heads	4	8
feed-forward width	256	1024
vocabulary		23 tokens, shared

The vocabulary is tiny because the task is: ten digits, two operators (`add`, `mul`), five syntax tokens, and six special markers. Positional information is learned, embeddings are tied between input and output, and there is no dropout.

The task both models were trained on

Problems look like `start 3 add 4 mul 2 add 1 ?` and the answer is 6, meaning 16 modulo 10. Three updates applied in order to one running value. Every step depends on the previous result, so the answer cannot be found without doing all three in sequence, and the difficulty comes purely from that depth rather than from having to search the input for anything. That is the whole point of the design: it isolates the thing a bigger model is supposed to be better at.

17,746 training problems, 500 for validation, 2,000 held out for the numbers reported here. Guessing scores 0.112.

How each was trained, step by step

Both models went through the same procedure independently, before any handoff existed.

1. **Learn the task with written reasoning.** The model is trained to produce the intermediate values as text and then the answer, which is ordinary next-token prediction.
2. **Replace the written steps with vectors, one stage at a time.** At stage one, the first written step is removed and a vector position takes its place; the model must carry that step internally. At stage two, two steps are gone. By the final stage the whole chain is vectors and only the answer is still written.

3. **Reset the optimiser at every stage boundary**, because the objective changes abruptly when a step disappears and the old momentum is actively unhelpful.

SETTING	VALUE
optimiser	AdamW, betas 0.9 / 0.95, weight decay 0.01
learning rate	3e-4, one-cycle schedule, 3% warmup
batch size	32
epochs per curriculum stage	3
gradient clipping	1.0
precision	float32

After this the small model scores 0.684 and the large model 0.997, both reasoning entirely in vectors. Those two numbers are the floor and the ceiling for everything that follows. It is worth noting the small model can reach 1.000 if allowed to write its reasoning out; the 0.684 is what it manages when forced to think silently, which is the regime the handoff operates in.

4. How the handoff works

1. The small model reads the question and keeps its cache. It will not read the question again.
2. Its final-layer hidden states are passed through the up-projector, which converts them from width 64 to width 256. Optionally only the last few states cross; because of how attention works those positions have already absorbed the whole question, so they carry it in compressed form.
3. The large model takes those vectors as if they were its own embeddings and runs several reasoning steps, each time feeding its own output hidden state back in as the next input. It never converts anything to words.
4. Those thought vectors go through the down-projector, back to width 64, and are inserted into the small model's context at positions reserved for them.
5. The small model writes the answer, reusing the cache from step one.

PIECE	WHAT IT IS	PARAMETERS
up-projector	Linear 64→256, GELU, Linear 256→256, output scale matching	0.041M
down-projector	the same in reverse, 256→64	0.041M
small-model adapter	LoRA rank 8	0.017M
large-model adapter	LoRA rank 8, starts at zero	0.017M

Everything trained for the handoff comes to roughly 0.12M parameters, about the size of the small model and 1.4% of the large one. Both backbones stay frozen.

5. How the handoff was trained

STAGE	WHAT IT TRAINS	WHAT IT OPTIMISES	STEPS
1	up-projector only	make the large model produce the same thoughts it would have produced reading the question itself, and keep its answer correct	2,200
2	down-projector and both adapters	answer correctness, end to end	1,650
3	everything, low learning rate	answer correctness	polish

Stage one exists to separate two failures that otherwise look identical. If the pipeline is broken, is it because the large model cannot read foreign vectors, or because the small model cannot use what comes back? Training and measuring the up-projector on its own answers that before the rest is built on top of it.

The detail that matters most

The large model has to be given a small adapter so it can learn to read the incoming vectors. Left completely frozen, it answers correctly only **26.8%** of the time from projected vectors, against **100%** from its own embeddings, and the whole pipeline stalls at 0.7485, which is no better than its control at 0.7415. With a rank-8 adapter it answers **99.4%** of the time from projected vectors and the pipeline reaches 0.9995. The adapter costs 0.017M parameters and does not change inference cost.

6. Results

CONFIGURATION	ACCURACY	FLOPS/QUERY	VS SMALL
small model, no reasoning at all	0.112	3.2M	0.8×
small model, writing its reasoning out	1.000	5.8M	1.4×
small model, reasoning silently (floor)	0.684	4.1M	1.0×
handoff, large model frozen	0.7485	242M	59.7×
<i>control</i> : untrained large model, frozen	0.7415	242M	59.7×
handoff, large model adapted	0.9995	242M	59.7×
<i>control</i> : untrained large model, adapted	0.7430	242M	59.7×
handoff, compressed to 4 vectors	0.8230	~120M	~30×
large model alone (ceiling)	0.997	317M	78.2×

What the controls rule out

- **It is not just extra parameters.** Swap the trained large model for an untrained one of identical shape and the score falls from 0.9995 to 0.7430, a gap of 0.2565. The projectors and adapters are unchanged, so the difference is the large model's learned computation.
- **It is not just more compute.** The small model given the same number of silent reasoning steps on its own scores 0.684.

- **Adaptation is the ingredient, not an incidental detail.** The frozen arm fails its own control while the adapted arm passes it, in the same experiment.

7. Where it does not work yet

The same architecture was tried with real pretrained models, Qwen2.5-0.5B handing off to Qwen2.5-7B, on two public benchmarks. It did not work.

PROSQA (N=500)	EXACT	HOTPOTQA (N=500)	EXACT
0.5B alone	0.734	0.5B alone	0.482
handoff, frozen	0.722	handoff	0.338
handoff, adapted	0.730	<i>control</i> : untrained receiver	0.362
<i>control</i> : untrained receiver	0.760	7B alone	0.596
7B alone	0.992		

On HotpotQA the handoff scored 0.024 below its own control, well within noise, so nothing crossed between the models. It also scored 0.144 below the small model working alone, which means the handoff did not merely fail, it made things worse. There are two candidate explanations and the compute window closed before they could be separated.

- **The projector was starved.** The version that works has 0.082M parameters and got 3,850 training steps. The HotpotQA version has 16.06M parameters and got 600. That is roughly a thousandfold less training per parameter. Published work fitting similar maps between language models uses 200M tokens and 36,000 iterations.
- **Pretrained models may be harder to connect.** Two models trained separately on the internet may not admit a clean correspondence the way two small models trained on the same narrow task do.

ProsQA was the wrong test regardless. The 0.5B scores 0.723 on three-hop problems and 0.833 on six-hop ones, so it does not degrade as reasoning gets deeper, which means it had no weakness for a larger model to cover.

8. The economics, and when they hold

The saving comes entirely from compressing the question before the large model reads it. That means the length of the input decides whether the idea pays. Figures below are for Qwen2.5-0.5B to 7B, sending 32 vectors.

BENCHMARK	INPUT LENGTH	COST VS SMALL ALONE	VS LARGE MODEL WRITING OUT REASONING
GSM8K	62 tokens	10.54×	2.7× cheaper
ProsQA	326 tokens	2.66×	7.6× cheaper
HotpotQA	1403 tokens	1.45×	12.3× cheaper

Sending every vector rather than a compressed handful costs 17.99× the small model and saves nothing at all, so compression is not an optimisation, it is the entire mechanism. It also costs accuracy: compressing to four vectors takes the small-scale result from 0.9995 down to 0.823.

9. Questions you are likely to get

Q. Is this just distillation?

No. Distillation copies capability into the small model's weights once, ahead of time, and then the large model is gone. Here the large model takes part in every single query, and the small model still cannot do the task alone, 0.684 against 0.9995. It is closer to a learned communication channel than to a training technique.

Q. Why not just run the large model?

On a long input the large model has to read all of it and then write out its reasoning. Here it reads a few dozen compressed vectors and writes nothing, which is where the $12\times$ comes from. The honest limit is that this only holds when the input is long relative to the reasoning. On short prompts the pipeline is ten times more expensive than the small model and the idea does not pay.

Q. How is this different from speculative decoding?

Speculative decoding has a small model guess tokens that the large model then checks, so the large model still reads everything and the answer is exactly what the large model would have said. Here the large model never sees or produces tokens at all, and the answer is the small model's. Different mechanism, different cost profile, different guarantee.

Q. How is this different from a mixture of experts?

A mixture of experts routes between parts of one model that were trained together and share a representation by construction. The difficulty here is that the two models were trained separately and share nothing, so the common format has to be learned from scratch. That is also why it is fragile.

Q. Why not have the small model just ask the large model in words?

That is the baseline being beaten. Words mean the large model reads a full prompt and writes a full answer. Interlat reports a $24\times$ latency reduction from replacing written agent messages with latent ones for exactly this reason. Vectors also carry more information per position than a token chosen from a fixed vocabulary.

Q. What if the two models use different tokenizers?

The experiments used a shared vocabulary on purpose, to remove that variable. Nothing about the architecture requires it, since the exchange happens in vector space rather than token space, but it is untested and would be the next thing to check.

Q. Doesn't training the large model defeat the purpose?

The adapter is 0.017M parameters, it is trained once, and it does not change what inference costs. The reason it is needed is that a frozen model has only ever seen its own embeddings; vectors arriving from elsewhere are outside that distribution and it answers at 26.8% from them. Two published papers agree. Interlat's ablation drops task success from 70.48 to 4.05 when their communication adapter is removed, and Cache-to-Cache reports 20.70% for replacing a receiver's internal state, recovered to 44.88% only by adding a residual path.

Q. How much training does the projector need?

This is the biggest open question. What is known: 0.082M parameters over 3,850 steps works, and 16.06M parameters over 600 steps fails badly. Published maps between language model representations use 200M tokens. The next experiment holds the model pair fixed and sweeps projector size against training budget.

Q. What happens when it fails?

It fails badly rather than gracefully. On HotpotQA the pipeline scored below the small model on its own, 0.338 against 0.482. An undertrained projector produces vectors the small model cannot interpret, and feeding it those is worse than feeding it nothing. Any real deployment would need a confidence check and a fallback to the small model alone.

Q. Aren't these models far too small to prove anything?

They are small, and the result should be read as a controlled demonstration rather than a product claim. What it establishes is that reasoning can cross between two independently trained models as vectors, across a $67\times$ size gap, with both controls passing. Whether that survives at real scale is unproven, and the attempts so far are written up above rather than left out.

Q. What is new here compared to Interlat and Cache-to-Cache?

Interlat sends latent messages between agents. Cache-to-Cache fuses one model's cached state into another's. Neither runs recurrent reasoning inside the receiver on the transmitted vectors and then sends the evolved result back for the original model to decode. Holding a reasoning trajectory across two latent spaces is the open problem this targets. The fair counterpoint is that both of those papers demonstrate their claims on real models and this one has not yet.

Q. What would make you abandon this?

The pipeline matching its untrained-receiver control at every scale attempted. That has now happened twice at 0.5B to 7B. If a properly trained projector at that scale still matches the control, the approach does not carry over to pretrained models and the small-scale result is a quirk of models shaped together on one narrow task.

10. Engineering notes

Six bugs turned up during the real-model runs, all of the kind that produce confident wrong numbers instead of crashing. Two are worth knowing if the conversation turns technical.

- **A $303\times$ scale mismatch.** The 0.5B's hidden states have a typical magnitude of 9.44; the 7B's embeddings sit at 0.0156. The projector's output scale started at 1.0, so it was handing the receiver vectors 64 times larger than anything it had seen. A published negative result on cross-model transfer reports the same failure at $80\times$.
- **Silently poisoned training.** In bfloat16, gradients over 1400-token inputs overflow before the forward pass does, so the loss looked finite while the weights were being corrupted. 485 of 500 batches were affected before a gradient check caught it.

11. What comes next

- Hold the model pair fixed and sweep projector size against training budget, to settle whether the 0.5B to 7B failure is starvation or something structural.
- Try adding the incoming vectors to the receiver's own state rather than replacing it, the approach that took Cache-to-Cache from 20.70% to 44.88%.
- Walk the scale up gradually: a few hundred million to 1B, then 0.5B to 7B, then 0.5B to 32B.
- Multi-hop question answering over documents is the right benchmark family, since it has both long inputs and a real gap between small and large models.

Two things not to claim

The 0.5B to 7B results do not work, so do not present them as if they do. And the arithmetic task is not a standard benchmark; it was built to control reasoning depth exactly, which is a reasonable design decision and lands much better said out loud than discovered.

References

- [1] S. Hao et al. Training Large Language Models to Reason in a Continuous Latent Space. arXiv:2412.06769, 2024.
- [2] Z. Du et al. Enabling Agents to Communicate Entirely in Latent Space. ACL 2026. arXiv:2511.09149.
- [3] T. Fu et al. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. arXiv:2510.03215, 2025.
- [4] A. Chen, J. Merullo, A. Stolfo, E. Pavlick. Transferring Linear Features Across Language Models With Model Stitching. NeurIPS 2025. arXiv:2506.06609.
- [5] A Negative Result on Cross-Model Activation Transfer in a Pythia Multi-Hop Setting. arXiv:2606.03280.